

Sub-module 7

BEGINNER'S GUIDE TO

Firmware on PicoRV32 + UART

Bare-metal bring-up, memory maps, UART I/O
and how firmware talks to hardware

From first boot to UART hello world — step by step

What We'll Cover

01 What is Firmware?

02 THIS Project Firmware File Tree

03 UART MMIO Map

04 Startup Code: crt0.S

05 `uart_putc + uart_getc`

06 Hello World → `uart_puts + echo loop`

07 Build System — actual fw/Makefile

08 Lab: `make fw` → `make soc`

What is Firmware?

Analogy

Think of firmware like the instructions printed on a microwave's control panel. The hardware (chip) is the microwave oven. The firmware is the tiny program baked inside that reads button presses and controls the heating element — it runs forever, doesn't need an OS, and talks directly to the chip's pins and registers.

What Firmware Is

- A C/assembly program compiled for a specific chip
- Runs directly on the CPU — no Linux, no Windows
- Stored in on-chip ROM or Flash memory
- The very first code the chip executes after reset

What Firmware Does

- Sets up the stack, clears memory, copies data
- Configures hardware peripherals (UART, GPIO...)
- Reads sensors, drives outputs in a tight loop
- Handles interrupts when events occur

Why RISC-V / PicoRV32?

- PicoRV32 = a tiny RISC-V CPU core for FPGAs
- AXI4-Lite bus connects CPU to peripherals
- Open-source: you can read every transistor!
- Perfect for learning SoC firmware from scratch

Project Firmware Structure — fw/ Directory

```
# fw/ directory
```

```
fw/
```

```
├─ main.c
```

```
├─ crt0.S
```

```
├─ link.ld
```

```
├─ Makefile
```

```
├─ bin2hex.py
```

```
├─ gen_rom_case.py
```

```
├─ firmware.elf
```

```
├─ firmware.bin
```

```
└─ → ../rtl/rom.hex
```

```
Build:    make fw
Inspect:  riscv32-unknown-elf-objdump -d
fw/firmware.elf
Verify:   cat rtl/rom.hex
```

main.c

Application

Greet 10 times then echo loop. Defines UART macros, uart_putc(), uart_getc(), uart_puts(), main().

crt0.S

Startup

Stack init (sp = 0x0001_00FC), BSS zero-fill, call main, trap on return. Runs before main().

link.ld

Linker Script

ROM@0x0, SRAM@0x10000, STACKADDR=0x10100. Controls where each section is placed in memory.

bin2hex.py

Tool

Converts firmware.bin to 4-byte-wide hex format for use with Verilog \$readmemh in ROM module.

gen_rom_case.py

Tool

Generates a synthesizable case-ROM Verilog module from rom.hex for ASIC/synthesis flows.

Firmware MMIO — PicoRV32 SoC Register Map

ROM 0x0000_0000

.text section (firmware instructions)

SRAM 0x0001_0000

stack + global variables

Stack Top 0x0001_00FC

set by STACKADDR in link.ld

UART TX 0x1000_0000

Write → [7:0] = byte to transmit

UART RX 0x1000_0004

Read → [7:0] = byte, [8]=rx_valid

UART STATUS 0x1000_0008

Read → [1]=rx_buf_valid

```
/* fw/main.c — hardware register definitions */
```

```
#define UART_TX      (*(volatile char*)0x10000000)
```

```
#define UART_RX      (*(volatile char*)0x10000004)
```

```
#define UART_STATUS (*(volatile int*) 0x10000008)
```

volatile → tells the compiler:

- DO NOT cache this read in a register
- DO NOT reorder or optimize away writes

✓ **No uart_init() needed**

uart_axi.v auto-enables on reset. Baud rate is compile-time fixed — zero runtime init.

↺ **AXI Backpressure**

When TX buf full: AWREADY/WREADY go LOW, CPU bridge stalls — no busy-wait loop needed.

Startup Code — Before main() • crt0.S

```
# fw/crt0.S — startup code

.section .text
.global _start
_start:

# 1. Init stack pointer
LUI sp, 0x10          # sp = 0x10000
ADDI sp, sp, 0xFC     # sp = 0x100FC
#    → STACKADDR = 0x0001_00FC

# 2. Zero BSS segment
LA a0, __bss_start
LA a1, __bss_end
bss_loop:
SW zero, 0(a0)
ADDI a0, a0, 4
BLT a0, a1, bss_loop

# 3. Call main()
CALL main

_exit: J _exit # trap on return
```

1

Reset vector

PicoRV32 jumps to 0x0000_0000 (PROGADDR_RESET) — first instruction in ROM

2

Stack init

LUI + ADDI sets sp = 0x0001_00FC
Defined by STACKADDR in fw/link.ld

3

BSS zero-fill

Clears BSS segment (globals = 0). In this project BSS = 0 bytes, but the pattern is critical.

4

Call main()

CALL main — firmware application starts. uart_puts() runs, then echo loop begins.

5

Trap on return

J _exit — infinite loop. main() should never return; this is a safety net.

uart_putc + uart_getc — Project API (no FIFO bit)

```
/* fw/main.c — UART API */

/* — Register macros — */

#define UART_TX      (*(volatile char*)0x10000000)
#define UART_RX      (*(volatile char*)0x10000004)
#define UART_STATUS (*(volatile int*) 0x10000008)

/* — Transmit one byte — */
static void uart_putc(char c) {
    UART_TX = c;
    /* AXI backpressure stalls CPU if buf full */
    /* NO busy-wait loop needed! */
}

/* — Receive one byte — */
static char uart_getc(void) {
    /* Poll STATUS bit[1] = rx_buf_valid */
    while (!(UART_STATUS & 0x2));
    return UART_RX; /* read clears rx_buf_valid */
}
```

Why no busy-wait in uart_putc?

When the TX buffer is full, uart_axi.v asserts backpressure: AWREADY and WREADY go LOW. The AXI bridge automatically stalls the CPU write — no software polling required.

Why poll STATUS in uart_getc?

ENABLE_IRQ=0 in this PicoRV32 config — no interrupt support. rx_buf_valid (STATUS bit[1]) must be polled. Reading UART_RX automatically clears the flag.

SiFive vs THIS project

SiFive uses bit[31] (FIFO full flag) + explicit polling. THIS project uses AXI backpressure — uart_putc is one line, not a while-loop wrapper.

Hello World → uart_puts + Echo Loop

```
/* fw/main.c – complete application */

static void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}

int main(void) {

    /* Send greeting 10 times */
    for (int i = 0; i < 10; i++)
        uart_puts("Hello Deepak from Nielit!\n");

    /* Echo loop – receive and retransmit */
    while (1)
        uart_putc(uart_getc());

    return 0; /* never reached */
}
```

Execution Flow

crt0.S

→ Stack + BSS → call main()

main()

→ uart_puts × 10 greeting

echo loop

→ uart_getc() → uart_putc()

Testbench Verification (tb/tb_top.v)

```
// Verify 10 greetings:
expect: "Hello Deepak from Nielit!"
// Repeat × 10 ✓

// Send PING, expect echo:
uart.send("PING");
uart.expect("PING"); // ✓

$display("TEST PASSED");
```

Build System — fw/Makefile

```
# fw/Makefile — actual project build
```

```
CC      = riscv32-unknown-elf-gcc
OBJCOPY = riscv32-unknown-elf-objcopy
```

```
CFLAGS = -march=rv32i -mabi=ilp32 \
          -nostartfiles -nostdlib \
          -T link.ld -O2 -Wall
```

```
SOURCES = crt0.S main.c
```

```
all: firmware.elf firmware.bin rom.hex
```

```
firmware.elf: $(SOURCES)
$(CC) $(CFLAGS) -o $@ $(SOURCES)
```

```
firmware.bin: firmware.elf
$(OBJCOPY) -O binary $< $@
```

```
rom.hex: firmware.bin
python3 bin2hex.py firmware.bin \
> ../rtl/rom.hex
```

-march=rv32i

Base ISA only — no M/A/F extensions. Matches PicoRV32 config.

-mabi=ilp32

int/long/pointer = 32-bit. No hardware float registers.

-nostartfiles

Don't link GCC's crt0.o — we use our own crt0.S.

-nostdlib

Don't link libc/libm — bare-metal, no stdlib.

-T link.ld

Use our linker script for ROM/SRAM/stack layout.

Firmware Size:

```
firmware.elf: ~167 bytes total
.text: 132 bytes (~33 instructions)
.data: 35 bytes (hello string)
```

MMIO — How Firmware Talks to Hardware

```
/* soc.h — all hardware addresses in one place */

#define UART0_BASE    0x10013000u

/* Byte offsets from UART0_BASE */
#define UART_TXDATA    0x00 /* Write char to send */
#define UART_RXDATA    0x04 /* Read received char */
#define UART_STATUS    0x08 /* TX full / RX empty */
#define UART_BAUDDIV   0x0C /* Clock divider */
#define UART_CTRL      0x10 /* Enable TX / RX */

/* Pointer macro for register access */
#define UART_REG(off) \
    (*(volatile uint32_t*)(UART0_BASE+(off)))
```

```
UART_REG(UART_TXDATA) = 'H'; // Sends the letter H over
serial!
```

UART Register Map

TXDATA	0x00	Write = send byte
RXDATA	0x04	Read = received byte
STATUS	0x08	Bit0=TXFULL, Bit1=RXEMPTY
BAUDDIV	0x0C	Baud = $\text{clk} \div \text{BAUDDIV}$
CTRL	0x10	Bit0=TXEN, Bit1=RXEN

Lab Execution — Building and Running Firmware

1 Build Firmware

```
make fw
```

- riscv32-unknown-elf-gcc (compile + link)
- objcopy -O binary → firmware.bin
- python3 bin2hex.py → ../rtl/rom.hex

2 Inspect ELF

```
riscv32-unknown-elf-objdump -d fw/firmware.elf
```

- 00000000 <_start>:
- lui sp, 0x10 # sp = 0x10000
- addi sp, sp, -4 # sp = 0x0001_00FC

3 Run Simulation

```
make soc
```

- ROM loads rtl/rom.hex via \$readmemh
- PicoRV32 fetches from 0x0000_0000
- UART output verified by testbench

4 Confirm TEST PASSED

```
[Simulation log output]
```

- [ROM] loaded from plusarg: rtl/rom.hex
- [ROM] Loaded 42 words
- TEST PASSED